

Build: AI Trading Intelligence OS

You are a senior product architect, quantitative developer, full-stack engineer, UI/UX designer, cybersecurity engineer, and financial-data systems engineer.

Build a production-quality web application called **[FINAL BRAND NAME]**.

This is NOT a simple trading dashboard and NOT a TradingView clone.

The product is an **AI-powered Trading Intelligence OS** that combines market analysis, advanced charting, strategy building, backtesting, trading journaling, trader-performance analytics, psychology analysis, and prop-firm risk management into one premium platform.

The core product loop is:

Market → Analyze → Build Setup → Backtest → Trade → Journal → Review → Improve

1. Technology Architecture

Use:

- React
- TypeScript
- Vite
- Tailwind CSS
- Modern component architecture
- Supabase PostgreSQL
- Supabase Authentication
- Supabase Storage
- Supabase Realtime
- Python
- FastAPI
- WebSockets where appropriate
- TradingView Lightweight Charts
- Python-based quantitative backtesting engine
- REST APIs for external market-data providers
- Secure environment variables
- GitHub-ready project structure

Do NOT create a monolithic application.

Separate the system into:

1. Frontend
2. API/backend
3. Market-data layer
4. AI analysis layer

5. Backtesting engine
6. Database
7. Authentication
8. External integrations

Create clean service interfaces so market-data providers can be replaced later without rewriting the application.

2. Brand Direction

Create a completely original premium fintech identity.

Do NOT copy TradingView, MetaTrader, Robinhood, Binance, or any existing trading platform.

Visual direction:

- Premium
- Minimal
- Professional
- Dark-first
- High information density without clutter
- Excellent typography
- Strong visual hierarchy
- Subtle glass/metal surfaces
- Restrained accent color
- Professional financial-terminal feel
- Smooth micro-interactions
- Responsive desktop/tablet/mobile experience

Do not overuse gradients.

Do not use excessive neon.

Do not make the interface look like a gaming dashboard.

The interface should feel like a professional trading workstation.

3. Main Navigation

Create:

Dashboard Markets Charts AI Intelligence Strategy Lab Backtest Trade Journal Trader DNA Prop
Challenge Economic Calendar Settings

Keep navigation clean and compact.

4. Dashboard

Create a professional trading command center.

Include:

Portfolio overview

Today's P&L

Win rate

Current drawdown

Open positions

Watchlist

Market movers

AI market summary

Upcoming economic events

Recent trades

Trading performance

Risk status

Trader health/risk score

Do not overwhelm the dashboard.

Allow users to customize dashboard widgets.

5. Markets

Create a market explorer.

Initially support:

XAUUSD EURUSD GBPUSD USDJPY BTCUSD NASDAQ SPX US30

Each instrument should display:

Current price Percentage change Bid/ask where available Session Volatility Daily high Daily low Market status

Allow:

Search Favorites Watchlists Sorting Filtering

6. Advanced Chart Workspace

Use TradingView Lightweight Charts.

Support:

Candlestick Line Area

Timeframes:

1m 5m 15m 30m 1H 4H 1D 1W

Indicators:

EMA 9 EMA 15 EMA 50 EMA 200 RSI MACD ATR Volume

Drawing tools:

Trendline Horizontal level Rectangle Fibonacci Price measurement

Smart market-structure overlays:

BOS CHOCH Liquidity Order Block Fair Value Gap Swing High Swing Low Support Resistance

Make every overlay optional.

Allow users to save chart layouts.

7. AI Intelligence

Create an AI analysis workspace.

The AI must NOT simply generate random BUY/SELL predictions.

It must analyze structured market information.

Inputs:

Higher timeframe structure Current timeframe structure Price action Liquidity Momentum Volatility
Indicators Economic events News where available Historical setup performance

Output:

Market Bias Structure Liquidity condition Momentum Volatility Key levels Potential setup Invalidation
Risk/reward Reasons Reasons NOT to trade

Example:

Market Bias: Bullish

Setup: Liquidity Sweep + BOS

Entry Zone: Defined price range

Invalidation: Defined level

Target: Defined level

Risk/Reward: 1:2.8

Confidence: Use a transparent analytical score, not a claim of certainty.

Always explain WHY.

Never claim guaranteed profit.

8. Unique Feature — Setup Score

Create a proprietary-looking analytical score called:

SETUP SCORE

Score each setup using:

Higher timeframe structure Liquidity Entry structure Momentum Volatility News risk Historical strategy
performance

Example:

87 / 100

Display the individual components.

Do not pretend this score guarantees success.

9. Unique Feature — Why NOT To Trade

Create a dedicated AI decision module.

The system should identify when conditions are unfavorable.

Examples:

High-impact news approaching Poor risk/reward Conflicting timeframes Low liquidity Consolidation
Weak momentum Overextended price Invalid structure Trader has exceeded risk limits

Display:

NO TRADE

with clear explanations.

This should be one of the core differentiating features.

10. Trade Setup Builder

Create a setup calculator.

Inputs:

Asset Direction Entry Stop Loss Take Profit Account Size Risk Percentage

Calculate:

Position size Dollar risk Potential profit R:R Percentage risk Distance to SL Distance to TP

Allow users to save the setup.

11. Risk Calculator

Create:

Position size calculator Risk-per-trade calculator R:R calculator Drawdown calculator Compounding calculator Daily loss calculator

Make calculations deterministic and testable.

Do not use AI for mathematical calculations.

12. Strategy Lab

Create a visual strategy builder.

Users should be able to construct:

IF conditions AND / OR logic THEN action

Example:

Liquidity Sweep = TRUE AND BOS = TRUE AND RSI > 50 AND Trading Session = London

THEN:

BUY

SL: 1 ATR

TP: 2 ATR

Create a strategy JSON representation behind the scenes.

Allow users to save, duplicate, edit and version strategies.

13. Natural Language Strategy Builder

Allow users to type:

"Create a XAUUSD 15-minute strategy using liquidity sweeps, BOS and 1:2 risk reward during the London session."

AI should convert this into a structured strategy definition.

Do NOT directly execute arbitrary AI-generated code.

Validate the strategy schema before sending it to the backtesting engine.

14. Backtesting Engine

Build the backtesting system in Python.

Support:

Historical OHLC data Entry rules Exit rules Stop loss Take profit Position sizing Trading sessions
Transaction costs Spread assumptions Slippage assumptions

Output:

Total trades Win rate Profit factor Net return Maximum drawdown Average R Average win Average loss
Largest win Largest loss Winning streak Losing streak Sharpe ratio where appropriate Expectancy
Equity curve

Display:

Equity curve Drawdown curve Monthly performance Trade distribution Win/loss analysis Long/short
performance

Avoid look-ahead bias.

Clearly separate training/backtest data from future/out-of-sample testing.

15. Trade Journal

Every trade should be journalable.

Fields:

Asset Direction Entry Exit SL TP Position size Risk Strategy Timeframe Session Entry reason Exit reason
Emotion Confidence Screenshot Result R multiple

Allow users to upload screenshots.

16. Trader DNA

This is a major unique feature.

Analyze the user's historical trading behavior.

Generate:

Best asset Best timeframe Best session Best strategy Average R:R Win rate Average holding time Most
profitable setup Worst setup Overtrading tendency Revenge trading tendency FOMO tendency Risk
consistency Performance by session Performance by weekday Performance by asset Performance by
strategy

Create a visual:

TRADER DNA

Do not make medical or psychological diagnoses.

Frame behavioral insights as observations from the user's recorded trading activity.

17. AI Trade Review

After a trade is closed:

Analyze:

Was the setup valid? Did the trader follow the strategy? Was risk appropriate? Was the entry late? Was the exit early? Was there evidence of overtrading? Was the trade consistent with historical profitable setups?

Return:

What went well What went wrong What to improve Strategy adherence Risk adherence

18. Prop Challenge Mode

Allow users to create a simulated prop-firm account.

Inputs:

Account size Profit target Daily drawdown Maximum drawdown Minimum trading days

Display:

Current equity Current profit Target remaining Daily drawdown remaining Maximum drawdown remaining Progress Risk status

Create warnings when the trader approaches limits.

19. Economic Calendar

Display:

Event Currency Impact Forecast Previous Actual Time

Highlight high-impact events.

Allow users to filter:

USD EUR GBP JPY

Create warnings such as:

HIGH IMPACT EVENT APPROACHING

Never fabricate economic data.

20. Notifications

Support:

Price alerts Setup alerts Economic-event alerts Risk alerts Drawdown alerts Journal reminders

Use server-side logic for scheduled/triggered notifications.

21. Authentication

Use Supabase Auth.

Support:

Email/password Google login if configured

Create user profiles.

Use Row Level Security.

Users must only access their own:

Trades Journal Strategies Backtests Screenshots Trader DNA Prop accounts Settings

22. Database

Create normalized PostgreSQL tables for:

users profiles assets market_prices market_candles economic_events market_news watchlists strategies
strategy_conditions backtests backtest_trades trades trade_setups trade_screenshots trading_journal
performance_metrics trader_dna prop_accounts prop_rules prop_metrics ai_analysis notifications
subscriptions

Create proper indexes.

Use timestamps consistently.

Use UUID identifiers.

23. API Integration Architecture

Create provider interfaces.

MarketDataProvider EconomicCalendarProvider NewsProvider AIProvider

Use environment variables.

Never hardcode API keys.

Required environment variables:

SUPABASE_URL SUPABASE_ANON_KEY SUPABASE_SERVICE_ROLE_KEY

TWELVE_DATA_API_KEY

OPENAI_API_KEY

FINNHUB_API_KEY

Optional:

NEWS_API_KEY BROKER_API_KEY BROKER_API_SECRET

Create a `.env.example` file containing placeholders only.

Never place real secrets in source control.

24. Security

Implement:

Authentication Authorization Row Level Security Server-side secret storage Input validation API rate limiting Request validation CORS configuration Secure headers Error handling Audit logging for sensitive operations

Never expose private API keys to the browser.

25. Data Reliability

Never fabricate market prices.

If market data is unavailable:

Display:

DATA UNAVAILABLE

Do not generate fake prices.

Show data source and timestamp where appropriate.

Clearly distinguish:

Live Delayed Historical Simulated Backtested

26. AI Safety

AI must not claim:

Guaranteed profits Guaranteed win rate Guaranteed predictions

AI output should be framed as:

analysis scenario potential setup risk assessment historical observation

The platform must clearly state that trading involves financial risk.

27. UI Pages

Create polished pages:

1. Landing Page
2. Login
3. Signup
4. Dashboard
5. Markets
6. Chart Workspace
7. AI Intelligence
8. Strategy Lab
9. Backtest
10. Trade Journal
11. Trader DNA

- 12. Prop Challenge
 - 13. Economic Calendar
 - 14. Settings
-

28. Landing Page

Hero:

"Trade with intelligence. Improve with evidence."

Supporting message:

"One workspace for market analysis, strategy testing, trading journals and AI-powered performance intelligence."

CTA:

Explore Platform

Secondary CTA:

View Demo

Include:

AI Intelligence Advanced Charts Strategy Lab Backtesting Trader DNA Prop Risk Control

Do not use generic stock-market clichés.

29. Responsive Design

Desktop should be the primary trading experience.

Support:

1920px 1440px 1280px 1024px 768px Mobile

Mobile should become a simplified trading interface rather than squeezing the desktop dashboard onto a phone.

30. Performance

Optimize:

Chart rendering Realtime updates Database queries API requests Caching Pagination Lazy loading

Do not request full historical datasets repeatedly.

31. Error Handling

Every external API must have:

Timeout Retry strategy Fallback handling Rate-limit handling Clear error state

Never crash the dashboard because one external provider is unavailable.

32. Testing

Create:

Unit tests API tests Backtesting tests Calculation tests Authentication tests Database security tests UI tests

Especially test:

Position sizing R:R Drawdown Backtesting Strategy execution Date/time handling Market sessions

33. Development Method

Do NOT generate the entire product blindly in one step.

First create:

1. Architecture
2. Database schema
3. API contracts
4. Component structure
5. UI design system
6. Development roadmap

Then implement incrementally.

After each major module:

Run tests.

Start the application.

Use browser testing.

Inspect errors.

Fix issues.

Do not move to the next major module if the current module is broken.

34. Development phases

PHASE 1

Foundation

- React
- TypeScript
- Tailwind
- Supabase
- Authentication
- Database
- Design system

PHASE 2

Trading Dashboard

- Markets
- Watchlist
- Chart
- Market data

PHASE 3

Trading Intelligence

- AI analysis
- Setup Score
- Why NOT To Trade

PHASE 4

Strategy Lab

- Visual strategy builder
- Strategy JSON
- Natural language strategy creation

PHASE 5

Backtesting

- Python engine
- Historical data
- Results
- Equity curve

PHASE 6

Journal

- Trade recording
- Screenshots
- Performance analytics

PHASE 7

Trader DNA

- Behavioral analytics
- Strategy performance
- AI review

PHASE 8

Prop Challenge

- Drawdown monitoring
- Risk limits
- Challenge dashboard

PHASE 9

Production

- Security
- Monitoring
- Rate limiting
- Performance optimization
- Deployment

35. Critical instruction

Do NOT build fake functionality.

Do NOT use fake market prices.

Do NOT use fake AI analysis presented as real.

Do NOT use mock backtest results in the production interface.

If an API is not connected, clearly show a setup state.

Build real underlying functionality.

36. Final quality standard

The finished product should feel like a premium professional trading workstation rather than a student project.

Prioritize:

Accuracy Performance Security Usability Visual hierarchy Data integrity Extensibility

The most important product principle is:

Help traders make better decisions through evidence, structured analysis and self-review — not promises of guaranteed profits.